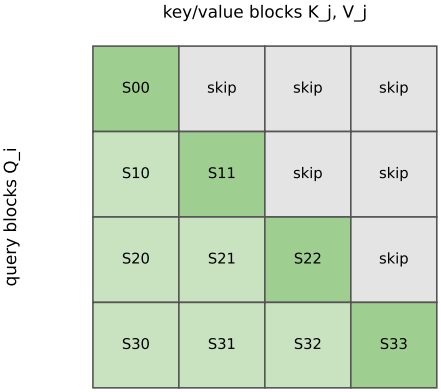

For one query block Q_i sweep key blocks j = 0..i, keeping per query row
m = running max, l = running sum of exp, o = running unnormalised output
S = Q_i K_j^T / sqrt(d) (+ causal mask)
m' = max(m, rowmax S), a = exp(m - m'), P = exp(S - m')
l' = a * l + rowsum(P), o' = a * o + P V_j ; at the end: output = o / l

Memory: only one B_q x B_k tile
of scores exists at a time
(never the full L x L matrix):
59-158x less extra memory
at L = 4096

Exact softmax attention
(no approximation):
max error 4e-5 vs float64 (fp32).
A plain PyTorch loop is slower
than the dense and fused versions